Simulating a Simple Cache Controller FSM

Name: Najmus Sakib
ID: 230041149

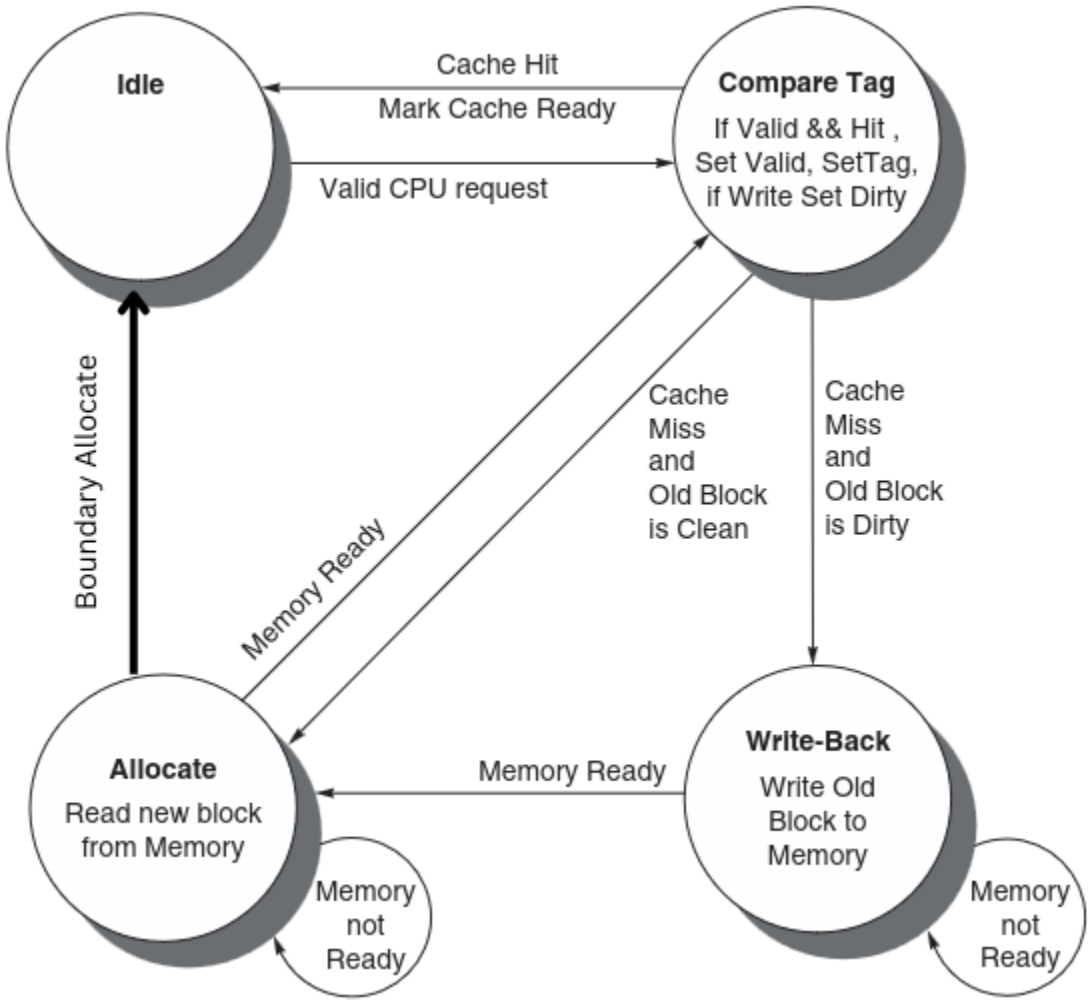
Name: Nuhiat Arefin
ID: 230041147

Name: Mehedul Hashan Prodhan
ID: 23004116

Submission Date: 16/04/2026

Git Repo: https://github.com/Afsan-z47/Cache_controller

FSM DIAGRAM:




```rust
fn tick(
    &mut self,
    inst: &Instruction,
    cache: &mut Cache,
    memory: &mut [u8],
    address_size: u32,
) {
    match self.state {
        State::IDLE => {}
        State::COMPARE_TAG => self.compare_tag(inst, cache),
        State::WRITE_BACK => self.write_back(inst, cache, memory, address_size),
        State::ALLOCATE => self.allocate(inst, cache, memory, address_size),
    }
}
```

Features:

- Step by step execution
- Cache dumps
- Simulate N-way set associative caches
- Simulate with LB, LH, LW, SB, SH, SW, NOP, INVALID instruction execution
- Handles cache boundary value access where the value is made up of 2 parts from the memory

Architecture:

The architecture of the software includes 3 main components:

- Instruction Parser
- Cache
- Cache Controller

Instruction Parser: Each instructions consists of 3 parts:

- OPCODE
- ADDRESS
- Store VALUE [Its made up of bytes]

If they don't need their associate values its set to a default;

```
const ADDRESS_SIZE: u32 = 1 << 8;

#[derive(Debug)]
enum OPCODE {
    LW,
    LH,
    LB,
    SW,
    SH,
    SB,
    NOP,
    INVALID,
}

#[derive(Debug)]
struct Instruction {
    opcode: OPCODE,
    value: [u8; 4],
    addr: u32,
}
```

Cache:

```rust
#[derive(Clone, Debug)]
pub struct CacheLine {
    pub is_valid: bool,
    pub is_dirty: bool,
    pub tag: u32,
    pub blocks: Vec<u8>,
}

impl CacheLine {
    pub fn new(byte_offset: u32) -> Self {
        Self {
            is_valid: false,
            is_dirty: false,
            tag: 0,
            blocks: vec![0; 1 << byte_offset],
        }
    }
}

pub struct Cache {    ■ fields in this struct
    pub byte_offset_bits: u32,
    pub index_bits: u32,
    pub tag_bits: u32,    ■ fields `tag_bits` and `data_size` are never r
    pub data_size: u32, /* in bytes */    ■ fields `tag_bits` and `data_s
    pub associativity: u32,
    pub cache_lines: Vec<Vec<CacheLine>>,
}
```

The cache is made up of:

- **Cache lines amounting the number of indexes and each multiplied by their associativity.**

```rust
impl Cache {    ■ method in this implementation
    pub fn new(byte_offset: u32, index: u32, address_size: u32, associativity: u32) -> Self {
        assert!(
            associativity.is_power_of_two(),
            "Associativity must be power of 2"
        );
        let sets = 1usize << index;
        Self {
            byte_offset_bits: byte_offset,
            index_bits: index,
            tag_bits: address_size - index - byte_offset,
            data_size: (1 << byte_offset) * (1 << index) * associativity,
            associativity: associativity,
            cache_lines: vec![vec![CacheLine::new(byte_offset); sets]; associativity as usize],
        }
    }
```

Cache_controller:
It has four main state. Another state Boundary fetch remain uncompleted and resides within allocation:

```rust
#[derive(PartialEq, Debug)]
enum State {
    IDLE,
    COMPARE_TAG,    variant `COMPARE_TAG` should have an upper camel case name  `#[warn(
    WRITE_BACK,    variant `WRITE_BACK` should have an upper camel case name
    ALLOCATE,
    //BOUNDARY_FETCH, //To be used to redirect boundary fetch from allocate
}

pub struct CacheController {
    state: State,
    unfinished: usize,
    bytes_done: usize,
    fetch_value: [u8; 4],

    pub hits: usize,
    pub misses: usize,
    pub write_backs: usize,
    pub double_allocations: usize,
}

impl CacheController {
    pub fn new() -> Self {
        CacheController {
            state: State::IDLE,
            unfinished: 0,
            bytes_done: 0,
            fetch_value: [0u8; 4],
            hits: 0,
            misses: 0,
            write_backs: 0,
            double_allocations: 0,
        }                                                                    Completed car
```

It also holds information about hits, miss, write backs.

Important thing to look for is:

- **Unfinished -> holds the number of unfinished allocation**
- **Bytes_done -> holds the number of bytes written before boundary fetch or 2nd allocation**
- **Fetch_values -> Acts as a buffer to remember old values between two types of fetch, and constructs the word, then fetches it to CPU as a whole.**

Example:

```
afsan@fedora:~/class_programming_training/COA/cache_controller$ cargo run input.txt
```

```
Commands:
s : Run 1 instruction
x : Print hit/miss/writeback statistics
d : Dump valid cache lines
q : Exit (prints final stats + dump)
h : Show this message

[0/51  51 left]> s
  Executing [0/50]: Instruction { opcode: LW, value: [0, 0, 0, 0], addr: 0 }
[ MISS ] addr=0 index=0 tag=0 occupant_dirty=false
[ALLOC] addr=0 block_base=0 data=[0, 0, 0, 0]
[ HIT ] LW addr=0 -> val=[0, 0, 0, 0]

[1/51  50 left]> s
  Executing [1/50]: Instruction { opcode: LW, value: [0, 0, 0, 0], addr: 4 }
[ MISS ] addr=4 index=1 tag=0 occupant_dirty=false
[ALLOC] addr=4 block_base=4 data=[0, 0, 0, 0]
[ HIT ] LW addr=4 -> val=[0, 0, 0, 0]

[2/51  49 left]> s
  Executing [2/50]: Instruction { opcode: LW, value: [0, 0, 0, 0], addr: 8 }
[ MISS ] addr=8 index=2 tag=0 occupant_dirty=false
[ALLOC] addr=8 block_base=8 data=[0, 0, 0, 0]
[ HIT ] LW addr=8 -> val=[0, 0, 0, 0]

[3/51  48 left]> d

   Cache Dump
   Way   Set  Valid  Dirty     Tag  Data
 ------------------------------------------------------------------
     0     0   true  false       0  [0, 0, 0, 0]
     0     2   true  false       0  [0, 0, 0, 0]
     1     1   true  false       0  [0, 0, 0, 0]
------------------------------------------------------------------

[3/51  48 left]> x


          Cache Statistics
 -------------------------------------
   Hits               : 3
   Misses             : 3
   Write-Backs        : 0
   Boundary crossings : 0
   Hit Rate           : 50.00     %
-------------------------------------

[3/51  48 left]>
```

Note: the test file is lacking :(